
Towards Distributed, Asynchronous Reinforcement Learning on Consumer Compute

Gordon Chen
School of Computing
University of Connecticut
gbc24001@uconn.edu

Abstract

Deep learning research increasingly depends on access to large amounts of compute. At the same time, due to recent interest in deep learning, compute is becoming harder to access for student researchers and hobbyists. In this paper, we develop a distributed, asynchronous reinforcement learning protocol that can make use of idle, heterogeneous consumer compute over the internet. We use IMPALA’s V-Trace algorithm to correct for stale policies and show via ablations that it is necessary in order to make asynchronous training stable. We introduce trajectory compression and protocol optimizations that allow us to beat a tuned synchronous baseline, reaching the maximum reward on Pong faster in wall-clock time. We open-source our code on GitHub at github.com/gordonbchen/taming-impala and hope that this framework will allow students and researchers to do even more awesome things through consumer compute pooling.

1 Introduction

Due to the increased interest in deep learning, compute for student and hobbyist research is becoming harder to access. Both memory and GPUs are getting more expensive, and cloud compute providers frequently run out of GPU instances when demand is high. Models are getting bigger and experiments are getting more expensive, meaning that even relatively small research projects can be constrained by access to compute.

Simultaneously, a large amount of consumer compute, such as desktops and gaming PCs, sits around unused for most of the day. Many students and hobbyists often have desktops with decent hardware that spend much of the day idle. Alone, these machines are often not powerful enough for modern deep learning experiments, even though collectively they may represent a substantial pool of useful compute.

We believe that reinforcement learning is very well suited for distributed asynchronous training, even more so than classic supervised learning. In supervised learning, data is collected beforehand and during training there is really only optimization. On the other hand, reinforcement learning algorithms very naturally split between alternating rollout and policy / value function optimization phases. RL’s unique property that data is collected online as the model changes makes it ripe for asynchronous training, which increases update throughput by overlapping rollout and optimization phases at the cost of introducing the negative effects of stale data.

In this paper, we develop a distributed, asynchronous reinforcement learning protocol that allows multiple geographically scattered machines to contribute compute to a shared training run over the internet. Our protocol follows the general setup from IMPALA, with actor nodes that collect trajectories using local, often stale, copies of the policy network, and periodically send those trajectories to a central learner over the internet, and receive updated model parameters in return.

Building upon IMPALA, we introduce protocol optimizations such as rollout compression and a custom message format to reduce internet bandwidth constraints, allowing for higher update throughput. The flexible protocol we develop is not strongly tied to any particular neural network architecture or environment. Furthermore, in contrast to the within-datacenter, distributed training with homogeneous compute that IMPALA was meant for, our protocol is developed for unreliable, bandwidth-limited setups for compute pooling over the internet, and thus supports actors with heterogeneous compute capabilities (CPU and GPU), as well as resilience to actors leaving and joining the compute pool in the middle of a training run.

We thoroughly ablate our protocol, and find that IMPALA’s V-Trace is necessary to stabilize asynchronous training at the levels of policy lag our cross-internet protocol faces.

2 Background

In 2018, Espeholt et al. introduced IMPALA, an asynchronous reinforcement learning architecture that uses multiple actor nodes to roll out trajectories with a local, possibly stale behavior policy and sends collected trajectories to a central learner, which optimizes the policy and value function, and broadcasts updated model parameters back to actors [1]. Critically, the authors introduce V-trace, an algorithm that uses importance-sampling corrections to adapt policy-gradient updates to stale-policy rollouts [1]. We will now derive V-trace.

We first start with the vanilla policy gradient

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \Phi_t \right]$$

where $J(\theta)$ is the expected, discounted cumulative reward of a parameterized policy π_{θ} in some Markov Decision Process. Rollout trajectories are denoted τ , and are made up of states s_t , actions a_t , and rewards r_t from times $1..T$.

Here, Φ_t can represent the cumulative discounted return, rewards-to-go, or any general advantage estimate. For our purposes, we choose Φ_t to be the baseline-subtracted bootstrapped return.

$$\begin{aligned} \Phi_t &= v_t - V_{\phi}(s_t) \\ v_t &= \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'} + \gamma^{T-t} V_{\phi}(s_T) \end{aligned}$$

Here, v_t is the bootstrapped return, V_{ϕ} is our parameterized value-function baseline, and γ is the future reward discount factor.

This return is essentially the cumulative discounted rewards plus the bootstrapped discounted value of the last state. Equivalently, we can rewrite this bootstrapped return in terms of TD-errors δ_t as follows:

$$\begin{aligned} \delta_t &= r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t) \\ v_t &= V_{\phi}(s_t) + \sum_{t'=t}^{T-1} \gamma^{t'-t} \delta_{t'} \end{aligned}$$

Now we can see that this bootstrapped return is the value of the current state plus discounted TD-error correction terms.

However, in the context of asynchronous training, we have a problem. In our equation for the bootstrapped return, $V_{\phi}(s_t)$ is meant to represent the value of state s_t under the current learner policy, but the rewards and therefore the TD-error correction terms come from trajectories collected on actors using stale policies. This mismatch means that $V_{\phi}(s_t)$ is being trained to predict returns induced by stale policies.

Since we are sampling from a stale policy, this bootstrapped return actually estimates v_t^{μ} , the expected cumulative discounted return if we follow a stale policy μ , instead of what we want, which is

v_t , the expected cumulative discounted return if we follow the learner’s most updated policy π .

$$v_t^\mu = \mathbb{E}_{\tau \sim \mu} \left[V_\phi(s_t) + \sum_{t'=t}^{T-1} \gamma^{t'-t} \delta_{t'} \right]$$

$$v_t = \mathbb{E}_{\tau \sim \pi} \left[V_\phi(s_t) + \sum_{t'=t}^{T-1} \gamma^{t'-t} \delta_{t'} \right]$$

V-Trace uses importance sampling to correct the TD-error terms.

$$v_t = \mathbb{E}_{\tau \sim \mu} \left[V_\phi(s_t) + \sum_{t'=t}^{T-1} \gamma^{t'-t} \left(\prod_{i=t}^{t'} \rho_i \right) \delta_{t'} \right]$$

$$\rho_t = \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)}$$

Since the importance sampling weight $\rho_t = \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)}$ can blow up when $\mu(a_t|s_t) \ll \pi(a_t|s_t)$ and cause v_t to be a very high-variance approximation, the authors use truncated importance sampling weights. The authors also split importance weights into the weight for the latest TD-error, ρ_t and the weight for all previous TD-errors, c_t to give an extra degree of freedom for reward shaping. Note that despite this, the authors of IMPALA use $\rho_{\max} = c_{\max}$, so the split does not matter in practice.

$$v_t = V_\phi(s_t) + \sum_{t'=t}^{T-1} \gamma^{t'-t} \left(\prod_{i=t}^{t'-1} \hat{c}_i \right) \hat{\rho}_{t'} \delta_{t'}$$

$$\hat{\rho}_t = \min(\rho_t, \rho_{\max}) \quad \hat{c}_t = \min(\rho_t, c_{\max})$$

We can now see why IMPALA sends rollouts from actors to the learner instead of gradients like A3C. Importance sampling requires both the log probabilities of the stale actor policy and those of the current learner policy on the same sampled actions. In the standard IMPALA architecture, the learner can recompute current-policy log probabilities on the received trajectories directly. In A3C-like architectures, each actor performs optimization locally, so IMPALA’s learner-side importance sampling correction is not directly available.

3 Methodology

For all experiments, we use the Pong-v5 environment from EnvPool for Gymnasium-compatible parallel environments [5]. We use EnvPool’s default Atari wrapper parameters, including 4-frame stack, grayscale, 4-frame skip, and random no-ops at the start of each episode for stochasticity.

We use a convolutional actor-critic network as the agent. The agent has a shared feature extractor backbone and actor and critic heads. The design of the agent is heavily inspired by *The 37 Implementation Details of Proximal Policy Optimization* by Huang et al. [2].

As a synchronous baseline, we use a single learner process with vectorized environments. In addition to the usual suspects of learning rate and Adam β_1, β_2 , we tune the number of parallel environments, rollout steps, and optimization steps to minimize the amount of time until the agent reaches maximal reward on Pong.

We also train a local asynchronous baseline using multiprocessing actors that write rollouts to a shared queue. The learner process pops rollouts, does optimization steps, and writes the latest weights to a queue for each actor to get.

For our distributed asynchronous architecture, we use multiple actor processes that communicate via TCP over Tailscale to a single learner process. Each actor requests updated weights from the learner, collects multiple rollouts in parallel environments, and sends the rollouts to the learner. On the learner side, there is the main optimization thread which gets rollouts from a shared queue, and a connection handler thread that constantly listens for TCP connections from actors. For each actor,

the connection handler spawns an actor-handling thread that responds to weight requests and collects rollouts in the rollout queue.

Every TCP message in our protocol starts with a header made up of protocol magic bytes and the message type: a `GET_WEIGHTS` request from the actor, a `WEIGHTS` response from the learner, a `ROLLOUT` batch from the actor, or an `ACK`. `ACK` messages just have the protocol header.

`GET_WEIGHTS` requests consist of just the header and the version number of the actor’s policy. The learner updates a policy version count that increments every time an optimization step is done. That way, if the actor already has the learner’s current policy version, the learner will just send an empty `WEIGHTS` message back that tells the actor to continue, and we don’t have to send the network weights to the actor.

`WEIGHTS` messages contain the learner’s policy version as well as all of the agent’s weights. Since we use `pytorch` in this project, the actor weights are completely defined by the state dict, which maps a name to each weight tensor. To serialize each tensor, we store the name of the parameter’s `dtype`, the number of dimensions, the shape, and the flattened raw bytes of the tensor. To deserialize the agent’s state dict, the actor reads the weight’s name and reshapes the tensor’s raw bytes.

`ROLLOUT` messages contain the actor’s UUID to identify which actors are contributing the most rollouts, the local policy version, and the rollout, which consists of observations, actions, rewards, actor policy log probabilities, and a done mask. The actor’s local policy version is used by the learner to throw away any rollouts that are too stale, set again by a configurable threshold. This prevents extremely old rollout batches from causing training destabilization.

Naively sending `ROLLOUT` and `WEIGHTS` messages would cause the protocol to be very inefficient and mostly network-bandwidth bottlenecked. With our settings, a rollout of 32 timesteps, 16 parallel environments per actor, 4 frames stacked per observation, and 84x84 grayscale frames would send 14.45 MB of rollouts across the internet for every actor. Our protocol introduced two optimizations for this problem, frame-stack deduplication and lossless rollout compression. Since each observation stacks the last 3 frames and the newest frame together, every additional observation really only has 1 frame of new information. So then instead of sending all 4 frames for every observation, we can just send all 4 frames of the first observation and whenever the environment resets, and the last frame for every additional observation. Then, on the learner’s side, the learner will use the reset mask to restack all frames. Using frame deduplication, we lower the memory usage of a rollout to around 3.95 MB, a 3.7x reduction in memory usage. Furthermore, there is reason to suspect that the observation frames are highly compressible. In Pong, most pixels do not move, so most frames are identical to previous frames, give or take a couple pixels where the paddle or the ball has moved. This means that even the deduplicated frames likely have a lot of redundancy, which we take advantage of by running `lz4` compression on, reducing the memory usage to 90 KB at a 43.9x compression rate. After all of our optimizations, we get around 160.6x rollout compression, using only around 176 bytes per observation. We also use compression on model weights, reducing the serialized parameter size from 1.8 MB to 1.3 MB. We use a `COMPRESSED` flag in the protocol so that we have the option to leave a message uncompressed if trying to compress it would not reduce the message size by enough to justify the extra decompression time.

The main difference between our protocol and the existing IMPALA protocol is that ours is built for heterogeneous compute pooling. Whereas the IMPALA protocol is meant for large-scale training runs in a datacenter with actors all running on similar, stable hardware, our protocol explicitly addresses the issues of fault tolerance and extreme rollout staleness. Since our protocol is meant to allow researchers to share idle compute, we need to make our protocol resilient to actors leaving and joining the training run at any time. We handle this by having the learner’s connection manager spawn a new handler thread for every actor that joins the training run. When actors leave the network, the learner’s corresponding handler thread exits, and everything gracefully continues from the perspective of the learner’s main optimization thread. When the device rejoins the network, the learner simply considers the device as a new actor and adds a handler. The separation of responsibilities between the optimization thread and the actor handlers on the learner allows us to have this clean API that connects the optimization and actor handlers solely by the rollouts in the queue and the most updated learner agent weights. To handle extremely stale rollouts, which can occur if some actors are very slow or the TCP message gets delayed in transit, we also introduce a configurable

maximum policy lag hyperparameter that the learner uses to throw away potentially destabilizing rollouts. Furthermore, another potential problem when doing this kind of asynchronous communication is that if the learner somehow takes too long to do an update, either because of a problem on the learner device or just if there are too many actors, actors can end up filling the learner’s rollout queue completely. This is a problem because then by the time the learner performs optimization steps on each rollout, the rollouts in the back of the queue put there long ago by the overhang of actors will already be stale, and they will go to waste, along with the compute on the actors side it took to collect those rollouts and the compute the learner used decompressing, deserializing, and preparing the rollout for optimization. To alleviate this problem, learners send ACK messages in response to collecting rollouts from actors. Actors can only continue sampling rollouts after a learner ACKs their last rollout, which provides some backpressure for actors to not completely saturate the learner’s queue.

4 Results

We heavily tuned our synchronous and local asynchronous baselines to provide a fair comparison to our distributed, asynchronous protocol. For our synchronous baseline, we ran hyperparameter sweeps on learning rate, Adam β_1, β_2 , update steps per rollout, number of parallel environments, and rollout length, searching for the configuration that would minimize time for the agent to reach maximum (21) reward on Pong. We chose the metric of time to maximum reward on Pong because Pong is a relatively simple environment that we can iterate quickly on, and achieving a faster time to essentially solve Pong would demonstrate the speed and power of our protocol. For our local asynchronous baseline, in addition to the hyperparameters we tuned for the synchronous baseline, we also tuned the number of actor processes, as well as the max policy lag before the learner throws out a rollout. Our synchronous baseline beat the local asynchronous baseline with 2 actors. We hypothesize that the reason for this is that the local asynchronous baseline simply introduces too much overhead compared to the synchronous baseline and ran much slower. The machine we used for experiments was simply not powerful enough to handle the 2 actors while running the optimal number of environments in parallel. We tried decreasing the number of parallel environments running in each actor, but then the learner would have to wait for 2 rollout batches for the optimal rollout batch size, which negated any gains we got from rolling out in parallel. So for the local asynchronous baseline, we truly were compute constrained, which is exactly the situation this paper tries to address. If we could run more actors in parallel, increasing the rollout throughput, then we would be able to beat the synchronous baseline.

For our distributed asynchronous protocol, we ran the learner and a single actor on a desktop connected to 2 actors on a laptop via Tailscale. Compare this to the asynchronous baseline, where we were trying to run a learner and 2 actors on the same desktop. Using the distributed asynchronous protocol, we were able to beat our heavily tuned synchronous baseline, reaching maximum reward on Pong in 3.3 minutes, compared to the synchronous baseline, which took 5.1 minutes (Figure 1). So our distributed protocol was able to beat the synchronous baseline in terms of wall-clock time. These gains came exclusively from higher rollout and update throughput. Comparing the synchronous and distributed runs by environment steps, both reach maximum Pong reward at around 2M environment steps (Figure 2). This makes a lot of sense, as the gains from distributed training do not come from increased sample efficiency. In fact, one would even suspect that distributed training would be worse for sample efficiency because of the stale-policy problem. Instead, the gains from distributed training, the 1.5x speedup from our distributed training setup, come just from increased rollout and update throughput, overlapping rollouts on actor devices with optimization steps on the learner device and eliminating any idle waiting time between the two phases.

We also run ablations on our protocol, empirically testing which parts of V-Trace, if any, are necessary to stabilize the heavily asynchronous training we do when the distributed aspect of our protocol increases latency and policy lag. We try training without clipping the importance sampling weights, and jettisoning V-Trace importance sampling corrections entirely. Both perform noticeably worse compared to the distributed protocol, with no V-trace performing worse than no-clipping (Figure 3).

Looking at our logged metrics, we develop a hypothesis as to why this may be the case. During training, with the 3 actor setup described above, the average policy lag is around 3.5, meaning that rollouts are collected, on average, by a local agent policy 3.5 update steps stale. This is quite a

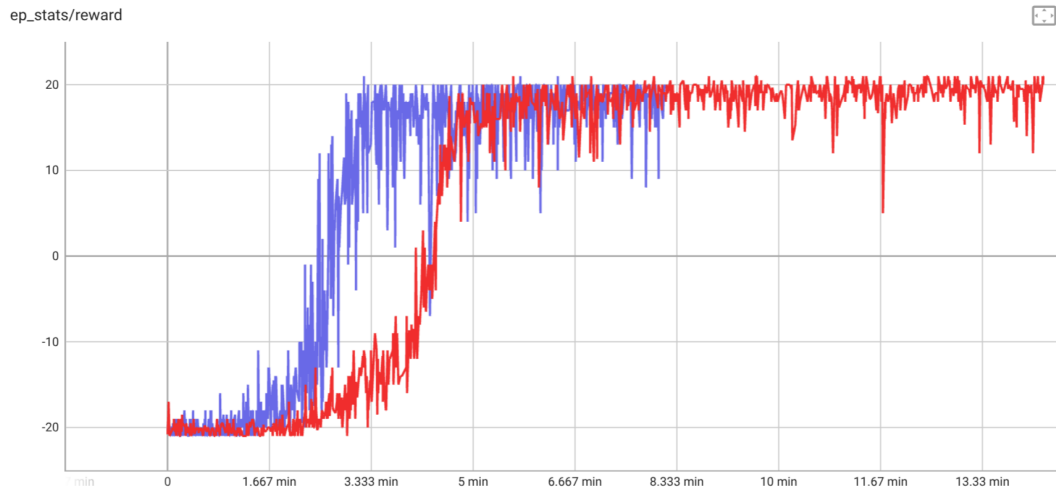


Figure 1: Wall-clock comparison of distributed protocol (blue) vs synchronous baseline (red).

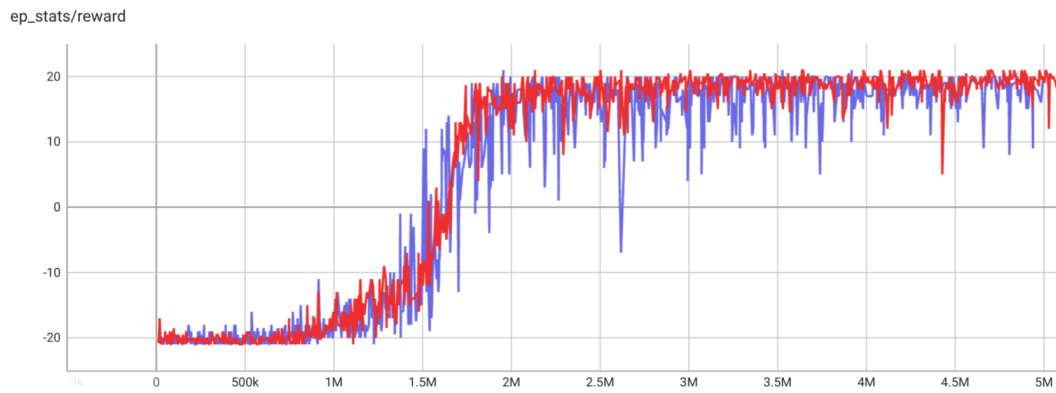


Figure 2: Env step comparison of distributed protocol (blue) vs synchronous baseline (red).

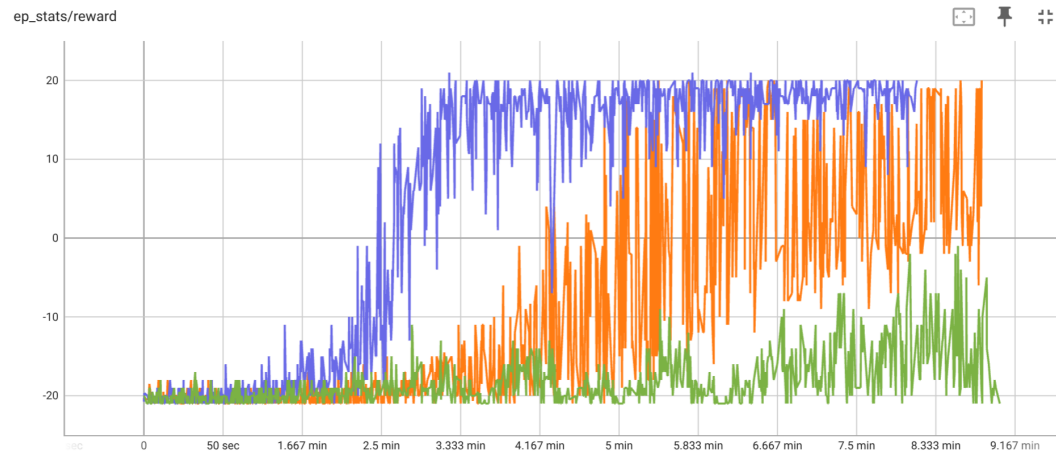


Figure 3: Reward: Dist (blue) vs No Ratio Clip (orange) vs No V-Trace (green).

bit. When rollouts are collected by policies this stale, the per-step action ratios can become quite large and unstable (Figure 4). Without clipping importance-sampling weights, these ratios can produce very high-variance value targets, destabilizing training and requiring large updates from both the value function, which must predict a wildly varying target, and the policy function, which must adapt to maximize that unstable value estimate. This instability is reflected in the gradient norm (Figure 5) exploding compared to the distributed protocol, which uses full V-trace. We also hypothesize that no V-Trace is even worse than just no-clip because while no-clip can produce high-variance value targets, the value targets are importance sampled and therefore are, in expectation, the correct value targets for the current learner policy. However, not using V-Trace at all means that the value targets are based off of very stale policies and are theoretically unsound and extremely unhelpful and destabilizing in practice.



Figure 4: Action ratios: Dist (blue) vs No Ratio Clip (orange) vs No V-Trace (green).

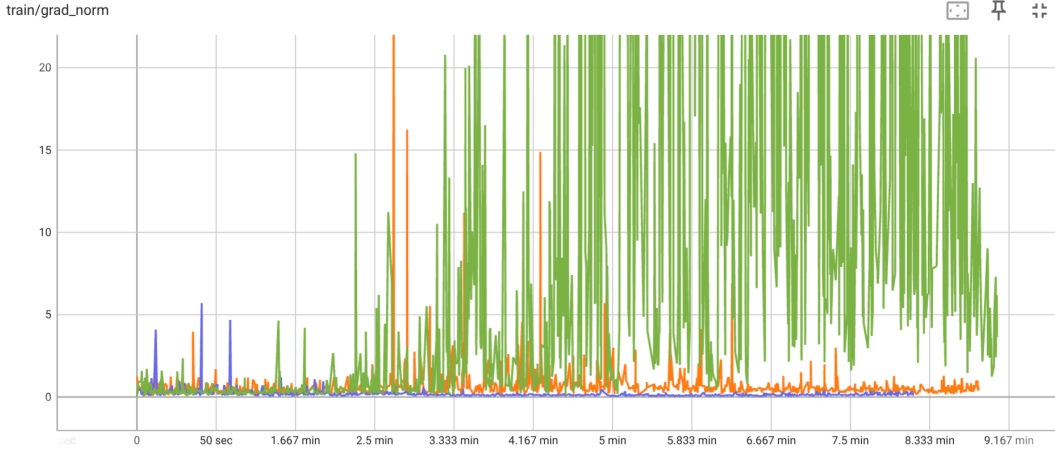


Figure 5: Grad Norms: Dist (blue) vs No Ratio Clip (orange) vs No V-Trace (green).

5 Limitations and Future Work

Despite showing that our distributed asynchronous protocol can reach maximum reward on Pong faster than a tuned synchronous baseline, the wall-clock gap of around 1.7 minutes is not an extremely substantial improvement. We see three ways to make the improvement between the distributed and synchronous protocols more substantial.

1. Heavier environments. Pong is a relatively fast environment. Each step does not take much time, so rolling out the environment is not really a bottleneck for RL training on Pong. If we used heavier environments where each environment step cost much more time and compute, then sampling rollouts could be a real bottleneck for training. Heavier environments would make it even more attractive to use asynchronous training, since synchronously the optimization phase would have to wait a long time for the rollout stage, but having multiple agents rolling out environments asynchronously and constantly streaming rollouts to the learner would allow for much higher update throughput.
2. Heavier policies. This is similar to having heavier environments. When policies get larger, sequential sampling, which requires forwarding the model once per rollout step and is what actors must do, gets much slower than batch evaluation, which just forwards the model a single time over the entire rollout and is what the learner does. This again makes rollouts the bottleneck and increases the improvement asynchronous training would give.
3. More actor devices. We only trained with a single additional actor device, but likely could have gotten even better rollout and update throughput had we increased the number of actor devices. At some point though, the single learner having to collect and run optimization on rollouts from all actors will become the bottleneck to more speed. Future work could try coming up with ways to scale beyond this bottleneck while trying to keep as much sample efficiency as possible.

Another limitation of this work is that the synchronous baseline we compare to is not the absolute strongest synchronous baseline we could use. In the future we would like to compare our distributed asynchronous protocol against PPO [3], the strongest synchronous baseline. Interesting future work could also try to mix PPO with IMPALA, using Generalized Advantage Estimation [4] instead of a bootstrapped return and including entropy rewards.

We would also like to scale our protocol beyond just a single private Tailnet and potentially develop a public learner service with GET weight and POST rollout endpoints so that anyone on the internet can contribute compute to a run. There are a couple different ways to achieve this. One option is where the service only accepts requests signed by a preshared key, ensuring that the nodes that join are trusted but also strongly limiting the scale of compute sharing. The other method would be to allow any nodes to join the network and contribute compute, but then there comes the problem of securing such a public network against malicious nodes that may spam the learner with garbage rollouts, or even more sinisterly, with rollouts that are slightly wrong and destabilize training. This is a very interesting problem that recent work on distributed RL for language models has started to explore [6].

While we only tested our protocol on Pong, we believe that distributed asynchronous RL could be very beneficial for reinforcement learning post-training on LLMs, since those often include long-horizon rollouts that make rollout generation a bottleneck for training [6].

6 Conclusion

In this paper, we develop a protocol for asynchronous, distributed reinforcement learning on heterogeneous consumer compute. We build off of the architecture introduced in IMPALA, and introduce compression techniques and protocol optimizations to make distributed training as fast as possible. We demonstrate its potential for speeding up RL training by beating a tuned synchronous baseline on Pong, and ablate it to empirically show that IMPALA’s V-Trace importance sampling corrections are essential for training stability.

We sincerely hope that the ideas from this paper can empower researchers to do more interesting experiments that they could not without compute pooling.

Acknowledgments

This paper was written for the final project of CSE 5820: Reinforcement Learning taught in Spring 2026 at UConn. Thank you to Professor Shi for teaching the class, it was a pleasure learning about this incredibly interesting field.

References

- [1] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.
- [2] Costa Huang, Alejandro Suarez, and others. The 37 Implementation Details of Proximal Policy Optimization. ICLR Blog Track, 2022. Available at <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>.
- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [4] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [5] Jiayi Weng, Min Lin, Shangdong Zhang, Bohang Zhou, Shuicheng Yan, and Weinan Zhang. EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine. *arXiv preprint arXiv:2206.10558*, 2022.
- [6] Prime Intellect. Prime-RL. GitHub repository, 2025. Available at <https://github.com/PrimeIntellect-ai/prime-rl>.